

Arquitectura: Patrones de diseño de Voz del ciudadano

Apellidos y nombres: Leon Gonzales Paul Fran

Código: 20232652A

El sistema de Voz del Ciudadano es un monolito que usa arquitectura en capas, se usa Java junto a su framework Spring Boot en el backend, y cuenta con un frontend usando html, css y javascript.

El sistema automatiza el proceso de las iniciativas legislativas ciudadanas, permite la creación de propuestas, la recolección de firmas digitales, el registro de comentarios y cuenta con un cierre criptográfico si se cumple con las condiciones para ello.(25 mil firmas)

- Arquitectura monolítica
- Arquitectura en capas

Controller

FirmaController.java

PropuestaController.java

Model

composite

ComponentePropuesta.java

ElementoTexto.java

ExpedienteDigital.java

flyweight

FirmaContexto.java

FirmaFlyweightCache.java

MetatadaPropuestaFlyweight.java

Repository

ElementoTextoRepository.java

FirmaRepository.java

PropuestaRepository.java

Service

facade

FachadaDespachoService.java

UtilidadCriptografica.java

proxy

FirmaService.java

FirmaServiceImpl.java

FirmaServiceProxy.java

VozDelCiudadanoApplication.java

- Frontend (html,js,css)

Patrones de diseño implementados

Nota: en los casos de uso esta explicado donde participan en el flujo acá se tiene un enfoque más de su objetivo, lo que logra y las capturas del código

Patrón Composite

Objetivo: Permite representar el expediente legislativo como una estructura jerárquica de elementos (propuesta + anexos + comentarios + modificaciones).

Componente

ComponentePropuesta

```
1  package com.vozdelciudadano.model.composite;
2
3  public interface ComponentePropuesta {
4      String obtenerContenido();
5  }
```

Hoja ElementoTexto

```
backend > src > main > java > com > vozdelciudadano > model > composite > ElementoTexto.java > ElementoTexto > contenido
1  package com.vozdelciudadano.model.composite;
2
3  import jakarta.persistence.*;
4  import java.time.LocalDateTime;
5
6  @Entity
7  @Table(name = "elementos_texto")
8  public class ElementoTexto implements ComponentePropuesta {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long id;
13
14     private String tipo; // "BASE", "ANEXO", "COMENTARIO", "MODIFICACION"
15
16     @Column(columnDefinition = "TEXT")
17     private String contenido;
18
19     private String autor;
20     private LocalDateTime fechaCreacion;
21
22     @Column(name = "propuesta_id")
23     private Long propuestaId;
24
25     public ElementoTexto() {}
26
27     public ElementoTexto(String tipo, String contenido, String autor, Long propuestaId) {
28         this.tipo = tipo;
29         this.contenido = contenido;
30         this.autor = autor;
31         this.propuestaId = propuestaId;
32         this.fechaCreacion = LocalDateTime.now();
33     }
34
35     @Override
36     public String obtenerContenido() {
37         return "[" + tipo + "] por " + autor + " (" + fechaCreacion + "): " + contenido + "\n";
38     }
39
40     // Getters y Setters para JPA
41     public Long getId() { return id; }
42     public void setId(Long id) { this.id = id; }
43     public String getTipo() { return tipo; }
44     public void setTipo(String tipo) { this.tipo = tipo; }
45     public String getContenido() { return contenido; }
46     public void setContenido(String contenido) { this.contenido = contenido; }
47     public String getAutor() { return autor; }
48     public void setAutor(String autor) { this.autor = autor; }
49     public LocalDateTime getFechaCreacion() { return fechaCreacion; }
```

Composite ExpedienteDigital

```
backend > src > main > java > com > vozdelciudadano > model > composite > ExpedienteDigital.java > ExpedienteDigital > setElementos(List<ComponentePropuesta>)
```

```
1 package com.vozdelciudadano.model.composite;
2
3 import jakarta.persistence.*;
4 import java.time.LocalDateTime;
5 import java.util.ArrayList;
6 import java.util.List;
7
8 @Entity
9 @Table(name = "propuestas")
10 public class ExpedienteDigital implements ComponentePropuesta {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     private String titulo;
17     private String estado; // "ACTIVA", "EXPIRADA", "CONGELADA", "DISTRIBUIDA_A_COMISIONES"
18     private int contadorFirmas;
19     private LocalDateTime fechaExpiracion;
20     private String hashSeguridad;
21
22     @Transient // JPA ignora este campo en mapeos simples, así que se cargara por repositorio
23     private List<ComponentePropuesta> elementos = new ArrayList<>();
24
25     public ExpedienteDigital() {}
26
27     public ExpedienteDigital(String titulo) {
28         this.titulo = titulo;
29         this.estado = "ACTIVA";
30         this.contadorFirmas = 0;
31         this.fechaExpiracion = LocalDateTime.now().plusDays(90);
32     }
33
34     public void agregarElemento(ComponentePropuesta componente) {
35         this.elementos.add(componente);
36     }
37
38     @Override
39     public String obtenerContenido() {
40         StringBuilder sb = new StringBuilder();
41         sb.append("EXPEDIENTE: ").append(titulo).append("\n");
42         sb.append("ID REGISTRO: ").append(id).append("\n");
43         sb.append("- CONTENIDO -").append("\n");
44         for (ComponentePropuesta elemento : elementos) {
45             sb.append(elemento.obtenerContenido());
46         }
47         return sb.toString();
48     }
49 }
```

Funciona así:

El expediente contiene una lista de elementos

Cada elemento implementa obtenerContenido()
Ser permite construir un expediente de manera dinámica

Con este patrón ahora se puede hacer un expediente flexible.

Evidencia en:

```
private List<ComponentePropuesta> elementos = new ArrayList<>();
```

```
public void agregarElemento(ComponentePropuesta componente) {  
    this.elementos.add(componente);  
}
```

```
@Override  
public String obtenerContenido() {  
    StringBuilder sb = new StringBuilder();  
    sb.append("EXPEDIENTE: ").append(titulo).append("\n");  
    sb.append("ID REGISTRO: ").append(id).append("\n");  
    sb.append("- CONTENIDO -").append("\n");  
    for (ComponentePropuesta elemento : elementos) {  
        sb.append(elemento.obtenerContenido());  
    }  
    return sb.toString();  
}
```

Patrón Flyweight

Objetivo: Reducir el consumo de memoria compartiendo la información común entre las firmas

Implementación:

Flyweight

MetadataPropuestaFlyweight

```

backend > src > main > java > com > vozdelciudadano > model > flyweight > J MetadataPropuestaFlyweight.java > MetadataPropuestaFlyweight
1  package com.vozdelciudadano.model.flyweight;
2
3  public class MetadataPropuestaFlyweight {
4      private final Long propuestaId;
5      private final String titulo;
6
7      public MetadataPropuestaFlyweight(Long propuestaId, String titulo) {
8          this.propuestaId = propuestaId;
9          this.titulo = titulo;
10     }
11
12     public Long getPropuestaId() { return propuestaId; }
13     public String getTitulo() { return titulo; }
14 }

```

FirmaFlyweightCache

```

backend > src > main > java > com > vozdelciudadano > model > flyweight > J FirmaFlyweightCache.java > ...
1  package com.vozdelciudadano.model.flyweight;
2
3  import java.util.HashMap;
4  import java.util.Map;
5
6
7  public class FirmaFlyweightCache {
8      private static final Map<Long,MetadataPropuestaFlyweight> cache = new HashMap<>();
9
10     public static MetadataPropuestaFlyweight obtenerFlyweight(Long propuestaId, String titulo){
11         if(!cache.containsKey(propuestaId)){
12             cache.put(propuestaId, new MetadataPropuestaFlyweight(propuestaId, titulo));
13         }
14         return cache.get(propuestaId);
15     }
16 }

```

FirmaContexto

```

backend > src > main > java > com > vozdelciudadano > model > flyweight > J FirmaContexto.java > FirmaContexto
1  package com.vozdelciudadano.model.flyweight;
2
3  import jakarta.persistence.*;
4  import java.time.LocalDateTime;
5
6  @Entity
7  @Table(name = "firmas")
8  public class FirmaContexto {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long id;
13
14     private Long ciudadanoId;
15     private Long propuestaId;
16     private LocalDateTime fechaFirma;
17
18     @Transient
19     private MetadataPropuestaFlyweight flyweight;
20
21     public FirmaContexto() {}
22
23     public FirmaContexto(Long ciudadanoId, Long propuestaId) {
24         this.ciudadanoId = ciudadanoId;
25         this.propuestaId = propuestaId;
26         this.fechaFirma = LocalDateTime.now();
27     }
28
29     public Long getId() { return id; }
30     public void setId(Long id) { this.id = id; }
31     public Long getCiudadanoId() { return ciudadanoId; }
32     public void setCiudadanoId(Long ciudadanoId) { this.ciudadanoId = ciudadanoId; }
33     public Long getPropuestaId() { return propuestaId; }
34     public void setPropuestaId(Long propuestaId) { this.propuestaId = propuestaId; }
35     public LocalDateTime getFechaFirma() { return fechaFirma; }
36     public void setFechaFirma(LocalDateTime fechaFirma) { this.fechaFirma = fechaFirma; }
37     public MetadataPropuestaFlyweight getFlyweight() { return flyweight; }
38     public void setFlyweight(MetadataPropuestaFlyweight flyweight) { this.flyweight = flyweight; }
39 }

```

Funcionamiento:

Datos compartidos: título de propuesta, id

Datos únicos: ciudadano, fecha de firma

El cache reutiliza las instancias existentes

Evidencia en

```

public static MetadataPropuestaFlyweight obtenerFlyweight(Long propuestaId, String titulo){
    if(!cache.containsKey(propuestaId)){
        cache.put(propuestaId, new MetadataPropuestaFlyweight(propuestaId, titulo));
    }
    return cache.get(propuestaId);
}

```

Patrón Proxy

Objetivo: Controlar el acceso al servicio de firmas evitando duplicados

Implementación:

Interfaz FirmaService

```
backend > src > main > java > com > vozdelciudadano > service > proxy > J FirmaService.java > ...
1  package com.vozdelciudadano.service.proxy;
2
3  import com.vozdelciudadano.model.flyweight.FirmaContexto;
4
5  //la interfaz en comun
6  public interface FirmaService {
7      FirmaContexto registrarFirma(Long ciudadanoId, Long propuestaId, String comentario);
8  }
```

ServicioReal FirmaServiceImpl

```
backend > src > main > java > com > vozdelciudadano > service > proxy > J FirmaServiceImpl.java > ...
16
17 //una implementacion tipica de la interfaz
18
19 @Service("firmaServiceReal")
20 public class FirmaServiceImpl implements FirmaService{
21
22     @Autowired
23     private PropuestaRepository propuestaRepository;
24
25     @Autowired
26     private ElementoTextoRepository elementoTextoRepository;
27
28     @Autowired
29     private FirmaRepository firmaRepository;
30
31     @Override
32     @Transactional
33     public FirmaContexto registrarFirma(Long ciudadanoId, Long propuestaId, String comentario) {
34         ExpedienteDigital propuesta = propuestaRepository.findById(propuestaId)
35             .orElseThrow(() -> new RuntimeException("Propuesta no encontrada"));
36
37         //1. La aplicacion de flyweight: se extrae el estado compartido de la cache
38         MetadataPropuestaFlyweight flyweight = FirmaFlyweightCache.obtenerFlyweight(propuesta.getId(), propuesta.getTitulo());
39
40         FirmaContexto nuevaFirma = new FirmaContexto(ciudadanoId, propuestaId);
41         nuevaFirma.setFlyweight(flyweight);
42
43         //2. la persistencia de la firma
44         firmaRepository.save(nuevaFirma);
45
46         //3. la aplicacion de composite, donde el comentario es una hoja que se agrega al expediente
47         if (comentario != null && !comentario.trim().isEmpty()) {
48             ElementoTexto hojaComentario = new ElementoTexto(tipo: "COMENTARIO", comentario, "Ciudadano_" + ciudadanoId, propuestaId);
49             elementoTextoRepository.save(hojaComentario);
50         }
51
52         //4. El contador se actualiza
53         propuesta.setContadorFirmas(propuesta.getContadorFirmas() + 1);
54         propuestaRepository.save(propuesta);
55
56         return nuevaFirma;
57     }
58
59 }
60 }
```

Proxy

FirmaServiceProxy

```
backend > src > main > java > com > vozdelciudadano > service > proxy > J FirmaServiceProxy.java > Language Support for Java(TM) by Red Hat > FirmaServiceProxy
1  package com.vozdelciudadano.service.proxy;
2
3  import com.vozdelciudadano.model.flyweight.FirmaContexto;
4  import com.vozdelciudadano.repository.FirmaRepository;
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.beans.factory.annotation.Qualifier;
7  import org.springframework.stereotype.Service;
8
9  @Service("firmaServiceProxy")
10 public class FirmaServiceProxy implements FirmaService {
11
12     // El Proxy guarda una referencia al objeto real
13     @Autowired
14     @Qualifier("firmaServiceReal")
15     private FirmaService firmaServiceReal;
16
17     @Autowired
18     private FirmaRepository firmaRepository;
19
20     @Override
21     public FirmaContexto registrarFirma(Long ciudadanoId, Long propuestaId, String comentario) {
22
23         // 1. Control de seguridad del Proxy pregunta si ya hay esa combinacion en la base de datos
24         boolean yaFirmo = firmaRepository.existsByCiudadanoIdAndPropuestaId(ciudadanoId, propuestaId);
25
26         if (yaFirmo) {
27             // El flujo se detiene si es un duplicado
28             throw new IllegalArgumentException("Acceso Denegado: El ciudadano " + ciudadanoId + " ya firmó esta propuesta.");
29         }
30
31         // 2. Si es una firma nueva, el Proxy delega la ejecución al objeto real
32         return firmaServiceReal.registrarFirma(ciudadanoId, propuestaId, comentario);
33     }
34 }
```

Funcionamiento:

El proxy intercepta la solicitud, valida si el ciudadano ya ha firmado; si es válido delega al servicio real

Evidencia en

```
boolean yaFirmo = firmaRepository.existsByCiudadanoIdAndPropuestaId(ciudadanoId, propuestaId);
```

Patrón Facade

Objetivo: Simplificar el proceso de cierre y distribución del expediente

Implementación:

FachadaDespachoService

```
backend > src > main > java > com > vozdelciudadano > service > facade > FachadaDespachoService.java > Language Support for Java(TM) by Red Hat > FachadaDe
12
13 @Service
14 public class FachadaDespachoService {
15
16     @Autowired
17     private PropuestaRepository propuestaRepository;
18
19     @Autowired
20     private ElementoTextoRepository elementoTextoRepository;
21
22     @Transactional
23     public ExpedienteDigital ejecutarCierreYDistribucion(Long propuestaId) {
24         // 1. Se recupera la propuesta base
25         ExpedienteDigital propuesta = propuestaRepository.findById(propuestaId)
26             .orElseThrow(() -> new RuntimeException("Propuesta no encontrada"));
27
28
29         // Caso de uso 4
30
31         // Paso 2: Bloqueo de seguridad (Cambio de estado inmediato a CONGELADA)
32         propuesta.setEstado(estado: "CONGELADA");
33         propuestaRepository.saveAndFlush(propuesta);
34
35         // Paso 3: Reconstrucción de Composite cargando sus hojas (Anexos, comentarios)
36         List<ElementoTexto> hojas = elementoTextoRepository.findByPropuestaId(propuestaId);
37         for (ElementoTexto hoja : hojas) {
38             propuesta.agregarElemento(hoja);
39         }
40
41         // Se extrae la cadena de texto lineal unificada desde el Composite
42         String contenidoCompleto = propuesta.obtenerContenido();
43
44         // Paso 4 y 5: Cierre criptográfico y persistencia del sello
45         String hashGenerado = UtilidadCriptografica.calcularSHA256(contenidoCompleto);
46         propuesta.setHashSeguridad(hashGenerado); // Se guarda la huella digital inmutable
47
48         // Paso 6 y 7: Distribución y actualización final
49         propuesta.setEstado(estado: "DISTRIBUIDA_A_COMISIONES");
50
51         return propuestaRepository.save(propuesta);
52     }
53 }
```

Funcionamiento:

Bloquea el expediente(congelado), reconstruye composite, genera el hash SHA-256 y cambia el estado a distribuido

Frontend:

Registrar Propuesta Normativa

Título de la Iniciativa

Ej. Ley de Protección de Áreas Verdes Urbanas

Texto Legal Base y Recursos de Soporte

Ingrese el texto del proyecto de ley junto con los enlaces de los documentos de soporte

Publicar Iniciativa

Catálogo de Iniciativas Disponibles

Seleccionar Propuesta Normativa

Seleccione una propuesta de la lista

Seleccione una propuesta de la lista

Ley de Proteccion de Areas verdes

Panel de Control de Auditoría (Facade)

ID	Iniciativa	Firmas	Estado	Sello Criptográfico (SHA-256)	Acciones
2	Ley de Proteccion de Areas verdes	0	ACTIVA	Abierto a firmas	Cierre Técnico
1	Ley de proteccion	2	DISTRIBUIDA_A_COMISIONES	cc44d06d217125769b5b5550...	Procesado